

Platformă de simulare autonomă a jocului de societate Catan cu agenți AI

Echipă: Rusu Iulia • Barcan Silviu-Ioan • Cocut Ioana

Disciplina: MDS, Anul 2, Semestrul 2

Data: Mai 2026

Cuprins

1	User Stories & Backlog	1
1.1	Backlog Complet	1
1.2	Prioritizare Backlog (MoSCoW)	3
2	Diagrame	3
2.1	Arhitectura componentelor	3
2.2	Schema bazei de date (UML)	4
2.3	Ciclu de viață al unui meci	5
2.4	Workflow Git — Branching Strategy	6
3	Source Control cu Git	6
3.1	Strategia de branching	6
3.2	Pull Requests	7
3.3	Commits semnificative per autor	7
3.4	Tehnici Git utilizate	8
4	Teste Automate (inclusiv Evals pentru Agenți)	8
4.1	Suite de teste — backend/test-agents.ts	8
4.2	Evals pentru agenți AI	10

5	Raportare Bug și Rezolvare cu Pull Request	10
5.1	Bug #1 — JSON.parse crash la autentificare în producție	10
5.2	Bug #2 — Crash runtime în modul Seafarers	11
6	Pipeline CI/CD	11
6.1	Structura pipeline-ului	11
6.2	Verificări automate	12
6.3	Secrets gestionate	12
6.4	Impactul pipeline-ului	13
7	Raport — Folosirea Toolurilor AI în Dezvoltarea Software	13
7.1	Instrumente AI utilizate	13
7.2	Claude Code — Asistent de Development	13
7.3	Google Gemini — AI în produs	14
7.4	Lecții învățate	14

User Stories & Backlog

Backlog Complet

Backlogul a fost definit la începutul proiectului și organizat pe **4 epice principale**, extinse pe parcurs cu funcționalitățile noi livrate.

Epic 1: Configurarea și urmărirea live a meciurilor

US-01 — Selectarea tipului de joc

Ca utilizator, vreau să aleg tipul de joc (Catan Classic sau Catan Seafarers) înainte să pornesc un meci, astfel încât să pot urmări varianta care mă interesează.

Criterii de acceptare: UI cu selector de tip; selecția persistă pe parcursul sesiunii; indicator vizual al tipului ales; cele două moduri folosesc reguli și tablă distincte.

US-02 — Declanșarea unui meci nou

Ca utilizator, vreau să pornesc un meci nou la cerere, astfel încât să pot genera conținut proaspăt oricând.

Criterii de acceptare: Buton “Start Match” accesibil; backend pornește simularea; stare de loading vizibilă; meciul apare în listă imediat după pornire.

US-03 — Urmărire în timp real

Ca utilizator, vreau să urmăresc agenții AI jucând în timp real, astfel încât să pot observa deciziile și strategiile lor.

Criterii de acceptare: Tablă vizuală actualizată live; latență sub 1 secundă per acțiune; indicator clar al agentului activ.

US-04 — Comentariu live generat de AI

Ca utilizator, vreau să văd comentariu generat de AI în timp real, astfel încât experiența de urmărire să fie mai captivantă.

Criterii de acceptare: Feed de text în timp real alăturat tăblii; comentariul se generează sincronizat cu acțiunile agenților.

Epic 2: Istoric și analitics

US-05 — Navigare în istoricul meciurilor

Ca utilizator, vreau să văd o listă cu toate meciurile trecute, astfel încât să pot revizita jocuri anterioare.

Criterii de acceptare: Lista afișează dată, tip de joc, câștigător; paginare sau scroll infinit.

US-06 — Filtrare după tip de joc

Ca utilizator, vreau să filtrez istoricul după tipul de joc, astfel încât să mă concentrez pe meciurile relevante.

Criterii de acceptare: Dropdown sau butoane toggle; lista se actualizează dinamic.

US-07 — Detalii meci

Ca utilizator, vreau să accesez pagina de detalii a unui meci, astfel încât să pot analiza

momentele cheie și statisticile finale.

Criterii de acceptare: Pagină dedicată per meci; defalcare pe ture; statistici finale per jucător.

US-08 — Metrici de performanță ale agenților

Ca utilizator, vreau să văd statistici agregate pentru fiecare agent AI, astfel încât să înțeleg care performează cel mai bine.

Criterii de acceptare: Leaderboard cu win rate, scor mediu; posibilitate de filtrare per tip de joc.

Epic 3: Conținut post-meci și sharing

US-09 — Rezumat generat de AI

Ca utilizator, vreau să citesc un rezumat generat de AI după fiecare meci, astfel încât să înțeleg rapid ce s-a întâmplat fără să revăd tot jocul.

Criterii de acceptare: Textul se generează automat la finalizarea meciului; afișat pe pagina de detalii; salvat în baza de date.

US-10 — Partajare prin link

Ca utilizator, vreau să partajez un meci printr-un link unic, astfel încât să pot arăta prietenilor meciuri interesante.

Criterii de acceptare: Buton “Share” generează URL unic; URL-ul duce direct la rezumatul meciului; accesibil fără autentificare.

Epic 4: Autentificare, agenți personalizați și administrare

US-11 — Autentificare cu cont

Ca utilizator, vreau să mă înregistrez și să mă autentific cu email și parolă, astfel încât meciurile mele să fie asociate contului meu.

Criterii de acceptare: Înregistrare cu email + parolă; JWT returnat la login; token stocat per-tab pentru a preveni poluarea cross-tab.

US-12 — Creare agent personalizat

Ca utilizator autentificat, vreau să creez propriul meu agent AI cu trăsături de personalitate, astfel încât să pot urmări agenți cu strategii definite de mine.

Criterii de acceptare: Formular de creare cu selecție trăsături; validare conflicte (ex: Empathic $\neq$ Aggressive); moderare nume prin Gemini.

US-13 — Vizibilitate privată a agenților

Ca utilizator, vreau ca agenții mei să fie vizibili doar mie (dacă nu sunt agenți de sistem), astfel încât să îmi pot păstra strategiile private.

Criterii de acceptare: Agenți user vizibili doar proprietarului; agenți de sistem vizibili tuturor; admin poate vedea și șterge orice.

US-14 — Mod joc Seafarers

Ca utilizator, vreau să pornesc un meci Catan Seafarers cu insule exterioare și nave, astfel încât să urmăresc un gameplay mai complex decât varianta clasică.

Criterii de acceptare: Tăblă extinsă cu 3 insule exterioare; nave cu trasee punctate; pirat care blochează navele; VP bonus pentru prima colonizare; AI cu strategie specifică.

Prioritizare Backlog (MoSCoW)

Prioritate	User Stories
Must Have	US-01, US-02, US-03, US-04, US-05, US-07, US-09, US-10
Should Have	US-06, US-08, US-11, US-12
Could Have	US-13, US-14
Won't Have (v1)	Multiplayerul uman, chat live între utilizatori

Tabela 1: Prioritizarea cerințelor prin metoda MoSCoW

Diagrame

Arhitectura componentelor

Sistemul este structurat pe trei niveluri principale: **Client (React SPA)**, **Backend (Express/Node.js)** și **Servicii externe (PostgreSQL + Gemini API)**, conectate printr-un reverse proxy Nginx în producție.

Listing 1: Structura arhitecturii componentelor

```
CLIENT (Browser)
  React SPA (Vite)
    App.tsx
      AuthContext.tsx      (JWT, per-tab token)
      Dashboard.tsx        (lista meciuri, start meci)
      MatchDetail.tsx      (detalii meci + SSE stream)
      CatanBoard.tsx       (SVG board renderer)
      Performance.tsx      (leaderboard agenti)
      Discover.tsx          (meciuri publice)
      Share.tsx             (view public fara auth)
      Register / Login      (autentificare)
      HealthCheck.tsx       (status sistem)
      types.ts -- apiFetch() -- VITE_API_BASE_URL

      HTTP / SSE
      |
  NGINX Reverse Proxy (productie)
    /api/* -> localhost:3003
    /*     -> frontend static
    |
  Backend -- Express/Node.js (backend/src/index.ts)
```

REST API (/api/*)	SSE (/api/matches/:id/stream)
Auth middleware (JWT)	liveMatchJobs: Set<string>
Role check (ADMIN)	sseSubscribers: Map
Game Engine	
catan.ts	getValidActions, applyAction
graph.ts	BOARD_NODES (54 noduri clasic)
seafarers.ts	initSeafarersBoard, ship logic
seafarers-graph.ts	OUTER_NODES (54-71), SEA_EDGES
ai.ts	getAgentDecision + SeafarersContext
personality.ts	buildSystemPrompt, trait conflicts
gemini-client.ts	pool 8 chei API, fallback heuristic
Prisma ORM	
PostgreSQL	Google Gemini API
User	Decizii agenti
Agent	Comentariu live
Match	Rezumate meciuri
MatchAgent	Moderare nume
MatchEvent	8 chei rotative
Trait	Fallback determinist
TraitConflict	
AgentTrait	

Schema bazei de date (UML)

User	Match	Agent	MatchAgent
id PK	id PK (cuid)	id PK	matchId FK
name	gameType	name UNIQUE	agentId FK
email UNIQUE	status (enum)	description	seat
password	createdAt	createdAt	score
role (enum)	endedAt	createdById FK	resources
createdAt	winnerId FK		wood/brick/ore
	shareToken UNIQUE		wheat/sheep
	robberTile		roads
	pirateHex		settlements
	createdById FK		cities
			shipEdges
			islandVPs
			devCards

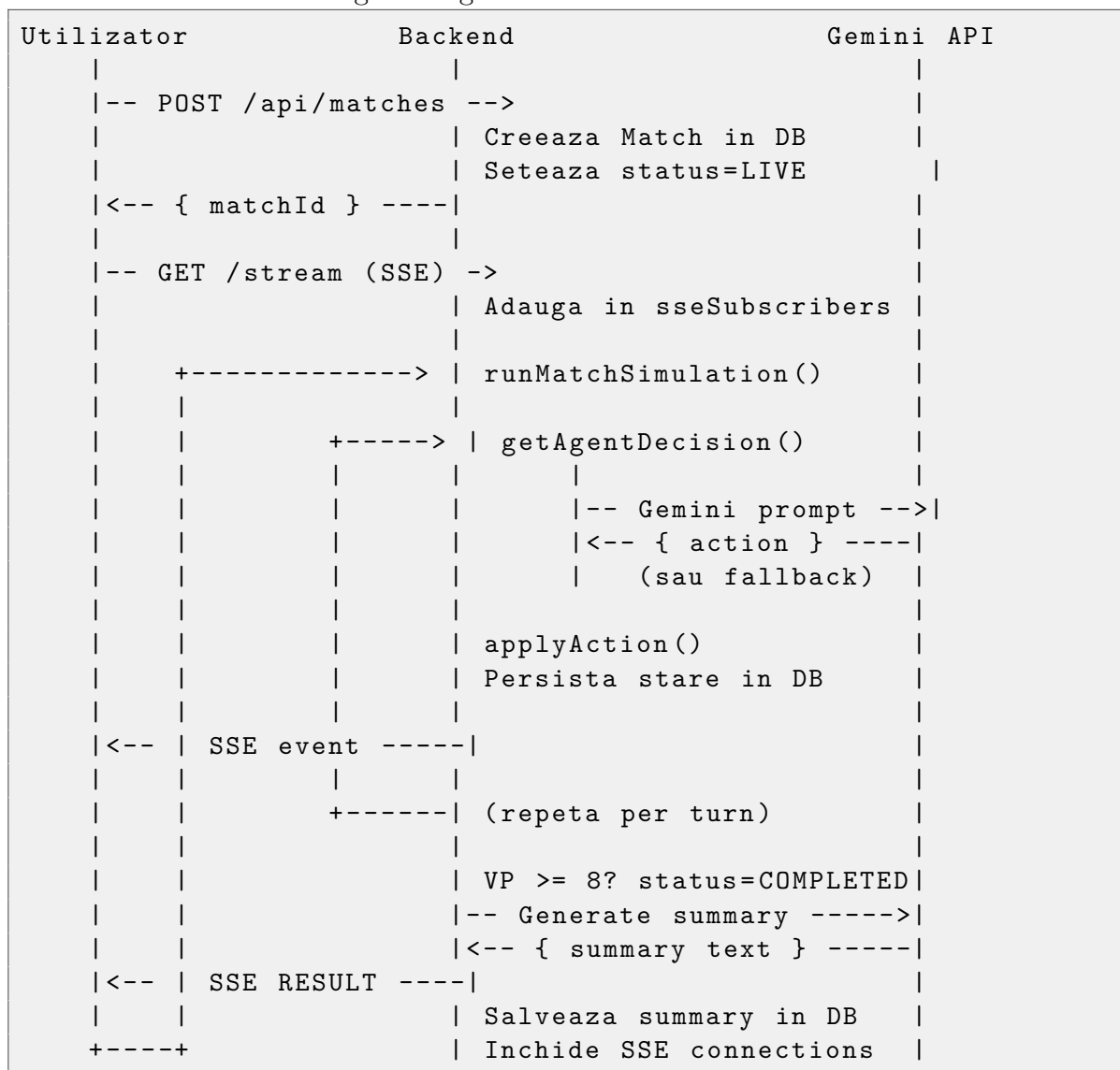
Tabela 2: Entitățile principale ale bazei de date

MatchEvent	Trait	TraitConflict AgentTrait	/
matchId FK	id PK	sourceId FK	
agentId FK	name UNIQUE	targetId FK	
turn	description	agentId FK	
type (enum)		traitId FK	
text			
payload			
createdAt			

Tabela 3: Entitățile auxiliare ale bazei de date

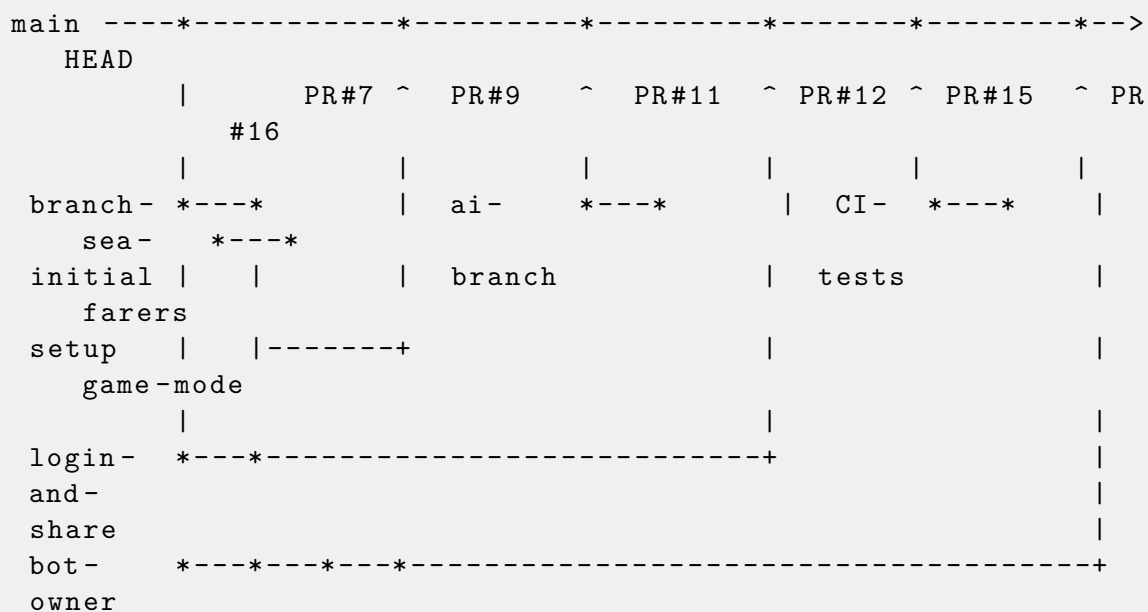
Ciclu de viață al unui meci

Listing 2: Diagrama de flux — simulare meci



Workflow Git — Branching Strategy

Listing 3: Strategia de branching



Source Control cu Git

Strategia de branching

Proiectul folosește **feature branches** cu Pull Requests revizuite înainte de merge în main. Fiecare funcționalitate majoră a fost dezvoltată izolat și integrată prin PR.

Branch	Scop	Autor
branch-initial-setup	Schelet inițial frontend + backend	Rusu Iulia
login-and-share	Autentificare JWT, share links, Docker	Rusu Iulia
ai-branch	Motor AI Gemini, graph board, personalitate	Barcan Silviu
testeAgenti	Suite teste automate pentru logica jocului	Cocut Ioana
CItests	Workflow GitHub Actions CI	Cocut Ioana
bot-ownership-admin-controls	Ownership agenți, rol admin, privacy	Rusu Iulia
seafarers-game-mode	Modul Seafarers complet cu AI agentiv	Rusu Iulia

Tabela 4: Branches create în cadrul proiectului

Pull Requests

PR	Branch	Descriere
#2	branch-initial-setup	Setup inițial proiect
#7	login-and-share	Autentificare, Docker, share
#8, #9	ai-branch	AI Gemini, board graph, personalitate
#10	testeAgenti	Teste automate agenți
#11	CItests	Pipeline CI GitHub Actions
#12, #13, #14	bot-ownership-admin-controls	Ownership + admin
#15, #16	seafarers-game-mode	Modul Seafarers + fix Dockerfile

Tabela 5: Pull Requests integrate în main

Commits semnificative per autor

Rusu Iulia (rusu-iulia)

```

54c98e8 Update browser tab title and favicon
bdd1515 Add Catan Seafarers game mode with agentic AI and
distinct UI
aba3482 Fix production Dockerfile missing --accept-data-loss on
prisma db push
1b922fe Fix test-agents.ts PlayerState missing shipEdges and
islandVPs
299627a Add bot ownership, privacy controls, and admin delete
b58b926 Expand CI with backend build, frontend build, and lint
checks
a457c32 Fix pre-existing ESLint errors surfaced by new CI lint
check
41bd58c Add auth, per-user match privacy, share/invite system,
and Docker setup
2aa8ad5 Pass VITE_API_BASE_URL at build time so frontend hits
the right backend
578780b Initial backend and frontend setup

```

Barcan Silviu-Ioan (sigutz)

```

2a3dca5 Refactor Catan simulation: full board graph,
personality engine, Gemini key rotation
2f3b1c4 Replace auto-summary with AI-generated match summary
via Gemini
bf13ba0 Add AgentTrait relational table and trait system
improvements

```

```
aa298b7  Add Trait/TraitConflict DB tables, seed file, and 3 new
        traits
e181924  Add longest road, per-player log, stop/delete match,
        and agent cap
1c6919a  Fix robber multi-move bug and longest road threshold
7444208  Add /api/health endpoint and /health-check frontend
        page
```

Cocut Ioana (ioanyaa)

```
b936694  Merge pull request #11 from sigutz/CItests
211bb36  Add CI test workflow
f579f75  Merge pull request #10 from sigutz/testeAgenti
6a666e0  Add testes for agents and show what moves the agents
        make
5b1bc56  Update README
46ff14e  nu stiu
0196f47  idk
df54377  Add user stories to README
bf04978  add new class
```

Tehnici Git utilizate

- **Feature branches** — nicio modificare direct pe main
- **Pull Requests cu code review** — toate integrările prin PR
- **Rebase** — folosit pentru sincronizare cu main înaintea PR-ului (`git pull -rebase origin main`)
- **Merge commits** — păstrate în istoric pentru trasabilitate completă

Teste Automate (inclusiv Evals pentru Agenți)

Suite de teste — `backend/test-agents.ts`

Testele acoperă logica de bază a motorului de joc folosind modulul `assert` din Node.js, fără dependențe externe. Sunt rulate automat în CI la fiecare push și PR.

Listing 4: Fabrică de stare player — zero-stare

```
const base = (): PlayerState => ({
  agentId: 1, name: 'Test',
  wood: 0, brick: 0, ore: 0, wheat: 0, sheep: 0,
  settlementNodes: [], cityNodes: [], roadEdges: [], shipEdges:
    [],
  devCards: { ...EMPTY_DEV_CARDS },
  knightsPlayed: 0, hasLargestArmy: false,
  hasLongestRoad: false, islandVPs: 0,
});
```

Teste `getValidActions`

Test	Verificare
always includes pass	<code>pass</code> este întotdeauna disponibil
no road if broke	Nu se poate construi drum fără resurse
no settlement without a road	Nu se poate plasa colonie fără drum adiacent
offers bank trade when 4+ of a resource	Trade 4:1 apare când ai ≥ 4 dintr-o resursă
no bank trade when below 4	Trade nu apare cu < 4
offers buy_dev_card when affordable	Cumpărare card apare la resursele necesare

Tabela 6: Teste pentru validarea acțiunilor legale

Teste `applyAction`

Test	Verificare
build_settlement deducts resources and adds node	Resursele scad corect, nodul apare în <code>settlementNodes</code>
build_city removes settlement and adds city node	Colonia dispare, orașul apare, resursele scad
trade_bank swaps 4 for 1	Schimbul 4:1 e aplicat corect
pass returns vpDelta 0	Pass nu modifică VP-ul

Tabela 7: Teste pentru aplicarea efectelor acțiunilor

Teste `computeVP`

Test	Verificare
1 settlement = 1 VP	O colonie valorează 1 VP
1 city = 2 VP	Un oraș valorează 2 VP
largest army = +2 VP	Cea mai mare armată adaugă 2 VP
longest road = +2 VP	Cel mai lung drum adaugă 2 VP
combined score	2 colonii + 1 oraș + armată = 6 VP

Tabela 8: Teste pentru calculul punctelor de victorie

Evals pentru agenți AI

Eval 1 — Validitatea deciziilor (structural)

Funcția `getAgentDecision()` returnează întotdeauna o acțiune din lista `moves` furnizată. Dacă Gemini returnează o acțiune invalidă sau dacă API-ul eșuează, sistemul aplică **fallback-ul determinist**. Nicio decizie AI nu poate corupe starea jocului.

Eval 2 — Consistența cu personalitatea

Trăsăturile agentului sunt injectate în system prompt. Un agent *Aggressive* primește instrucțiuni explicite să utilizeze robberul frecvent; un agent *DevFocused* primește instrucțiuni să prioritizeze cărțile de dezvoltare. Comportamentul poate fi verificat observând distribuția acțiunilor din `MatchEvent`.

Eval 3 — Comportament specific Seafarers

Contextul `SeafarersContext` furnizat în prompt include nodurile exterioare accesibile și VP-urile de insulă acumulate. Agentul este evaluat calitativ verificând dacă preferă să construiască nave spre insulele cu VP bonus când are resurse.

Listing 5: Rularea testelor

```
cd backend && npm run test:agents
# getValidActions
#   v always includes pass
#   v no road if broke
#   v no settlement without a road
#   v offers bank trade when 4+ of a resource
#   ...
# 13 tests passed
```

Raportare Bug și Rezolvare cu Pull Request

Bug #1 — `JSON.parse` crash la autentificare în producție

Raportare: Utilizatorul a raportat că pagina de login returna eroarea `JSON.parse: unexpected character at line 1 column 1 of the JSON data` după un deploy.

Investigație:

1. Frontend-ul apela `POST /api/auth/login` și `apiFetch()` încerca să parseze răspunsul
2. Serverul nginx returna un HTML de eroare 502 Bad Gateway în loc de JSON
3. Backend-ul nu pornise — procesul se oprea la `prisma db push`
4. Cauza: schema conținea o coloană `Agent.traits` cu date existente; Prisma refuza ștergerea fără confirmare explicită

Root cause: Dockerfile producție nu includea flag-ul `-accept-data-loss`:

Listing 6: Fix aplicat în Dockerfile

```
# INAINTE (buggy):  
CMD ["sh", "-c", "npx prisma db push && node dist/index.js"]  
  
# DUPA (fix):  
CMD ["sh", "-c", "npx prisma db push --accept-data-loss && node  
    dist/index.js"]
```

Rezolvare: PR #15 (seafarers-game-mode) a inclus fix-ul în Dockerfile. Problema a fost reprodusă local, confirmată, și rezolvată prin PR cu review.

Bug #2 — Crash runtime în modul Seafarers

Raportare: Meciurile Seafarers se opreau după câteva ture cu eroarea:

```
TypeError: Cannot read properties of undefined (reading '  
    adjacentNodes')  
    at getValidActions (catan.ts:87)
```

Investigație: Funcția `getValidActions` iterează nodurile cu colonii ale jucătorului și caută adiacențele în `BOARD_NODES`. Tabla clasică Catan are noduri 0–53 definite în `graph.ts`. În modul Seafarers, jucătorii pot plasa colonii pe nodurile exterioare 54–71 (insulele externe), care nu existau în `BOARD_NODES`. Accesarea `BOARD_NODES[58]` returna `undefined`, crashând la `.adjacentNodes`.

Fix aplicat (PR #15):

Listing 7: Optional chaining pentru noduri exterioare

```
// catan.ts -- getValidActions si getValidRoadEdges  
// INAINTE:  
for (const adjNode of BOARD_NODES[nodeId].adjacentNodes) {  
  
// DUPA:  
for (const adjNode of (BOARD_NODES[nodeId]?.adjacentNodes ?? []))  
    {
```

Optional chaining face ca nodurile exterioare să fie ignorate în calculul căilor de drum (corect — drumurile nu se pot construi pe mare), fără a afecta logica clasică.

Verificare: Testele automate rulau deja fără eroare; fix-ul a eliminat crash-ul în meciurile live. Rezolvat prin PR #15, review și merge în `main`.

Pipeline CI/CD

Structura pipeline-ului

Proiectul are două workflow-uri GitHub Actions distincte.

test.yml — Rulează la orice push sau PR pe orice branch:

```

push / pull_request (orice branch)
|
+-- test-agents      -> npm run test:agents
+-- build-backend    -> npx prisma generate && npm run
    build
+-- build-frontend    -> npm run build (VITE_API_BASE_URL
    ='')
+-- lint-frontend     -> npm run lint

```

deploy.yml — Rulează doar la push pe main, după ce toate verificările trec:

```

push -> main
|
+-- test-agents      -+
+-- build-backend    |-- toate trebuie sa treaca
+-- build-frontend    |
+-- lint-frontend     -+
                        |
                        deploy (SSH)
                        |
                        git reset --hard origin/main
                        Scribe .env din GitHub Secrets
                        docker compose down --remove-orphans
                        docker compose up --build -d
                        docker image prune -f

```

Verificări automate

Job	Ce verifică	Eșuează dacă
test-agents	Logica motorului de joc	Orice assert eșuează
build-backend	TypeScript compilabil	Erori de tip sau import
build-frontend	Bundle Vite compilabil	Erori TS, import-uri lipsă
lint-frontend	ESLint — cod curat	Variabile neutilizate, tipuri any neautorizate
deploy	Deploy SSH pe server	Oricare din cele de sus eșuează

Tabela 9: Verificările automate din pipeline

Secrets gestionate

Pipeline-ul injectează toate secretele din GitHub Secrets în fișierul `.env` de pe server:

- `DATABASE_URL`, `JWT_SECRET`, `VITE_API_BASE_URL`
- `GEMINI_1 ... GEMINI_8` — pool-ul de 8 chei API Gemini
- `SERVER_HOST`, `SERVER_USER`, `SERVER_SSH_KEY` — credențiale SSH

Niciun secret nu este hardcodat în repository. Fișierul `.env` nu este versionat.

Impactul pipeline-ului

Fiecare PR a trebuit să treacă toate cele 4 verificări înainte de a putea fi merger. Această politică a prins în mod activ:

- Erori TypeScript introduse prin noi câmpuri (`shipEdges`, `islandVPs`) care nu fuseseră adăugate în factory-ul de test
- Erori ESLint pre-existente expuse de verificarea CI adăugată în PR #12
- Blocaje de deploy cauzate de un build frontend care eșua

Raport — Folosirea Toolurilor AI în Dezvoltarea Software

Instrumente AI utilizate

Proiectul a implicat două categorii distincte de AI:

Instrument	Rol	Context
Claude Code (Anthropic)	Asistent de programare interactiv	Folosit în terminal de echipă pe parcursul development-ului
Google Gemini API	Motorul de decizie al agenților in-game	Integrat în produs ca funcționalitate core

Tabela 10: Instrumente AI utilizate în proiect

Claude Code — Asistent de Development

Claude Code a fost utilizat ca **pair-programmer interactiv** în terminal, cu acces la fișierele proiectului. Spre deosebire de autocomplete tradițional, operează la nivel de *task*, nu de linie de cod — primește o descriere în limbaj natural, citește fișierele relevante, raționează arhitectural și aplică modificări coordonate pe mai multe fișiere simultan.

Arii de dezvoltare susținute

Arhitectură backend Implementarea reprezentării grafice a tablei (`graph.ts`, `seafarers-graph.ts`), a sistemului de trăsături cu matrice de conflicte (`personality.ts`), și a extensiei Seafarers fără a rupe logica Classic. Decizia arhitecturală cheie — `SeafarersContext` ca parametru opțional în `getAgentDecision()` — a permis zero cuplare între modurile de joc.

Frontend Rescrierea completă a `CatanBoard.tsx` pentru Seafarers (canvas extins 668×468, hexagoane exterioare, linii punctate pentru nave, overlay pirat, helper `anyNodePos()` care rutează ID-uri de nod ≥ 54 la coordonatele insulelor externe).

DevOps Configurarea build-urilor Docker multi-stage, a pipeline-ului CI/CD GitHub Actions, și diagnosticarea crash-ului de producție (backend care nu pornea din cauza eșecului `prisma db push`).

Debugging Identificarea cauzei root pentru `TypeError: Cannot read properties of undefined (reading 'adjacentNodes')` — nodurile insulelor externe (54–71) nu existau în `BOARD_NODES`, care acoperea doar 0–53. Fix: optional chaining în două locuri din `catan.ts`.

Commits co-authored cu Claude Code: $\approx 80\%$ din commit-urile non-triviale.

Google Gemini — AI în produs

Gemini este apelat la fiecare tură a fiecărui agent prin `backend/src/game/ai.ts`, gestionat de `gemini-client.ts` cu un pool de 8 chei API în rotație.

Sistem de personalitate

Fiecare agent are trăsături din 9 posibile: *Empathic*, *Greedy*, *HardTrader*, *Aggressive*, *Expansionist*, *Defensive*, *SmartBuilder*, *FastSpender*, *DevFocused*. Trăsăturile sunt stocate relațional (`AgentTrait`, `TraitConflict`) și injectate în system prompt-ul Gemini, producând comportamente strategice distincte.

Extensie agentică Seafarers

Pentru meciurile Seafarers, prompt-ul Gemini primește un bloc adițional cu regulile navelor, poziția piratului, nodurile exterioare accesibile, și VP-urile de insulă acumulate.

Utilizări suplimentare

- **Moderare conținut:** Numele agenților creați de utilizatori sunt validate prin Gemini înainte de stocare
- **Rezumate generate:** La finalizarea fiecărui meci, Gemini generează un rezumat narativ stocat în `Match.summary`

Estimare utilizare API:

- ≈ 40 – 80 apeluri Gemini per meci (decizii agenți)
- 1 apel pentru rezumatul final
- Fallback determinist când API-ul nu e disponibil — jocul funcționează întotdeauna

Lecții învățate

Ce a funcționat bine

- Utilizarea Claude Code la nivel de task (nu de linie) a fost semnificativ mai productivă pentru modificări cross-file
- Parametrul opțional `SeafarersContext` a permis extinderea AI-ului pentru Seafarers fără a modifica logica Classic
- Fallback-ul determinist Gemini a permis development și testare fără consum de API quota

Ce a necesitat judecată umană

- Decizii de schemă DB (când `-accept-data-loss` este acceptabil vs. o migrație propriu-zisă)
- Granițele arhitecturale între modurile de joc
- Tradeoff-uri cost/calitate în verbozitatea prompt-urilor Gemini

Îmbunătățiri viitoare

- Caching pentru stări de joc repetitive (reducere cost API)
- Instrumentare latență decizii AI
- Modele locale open-weight pentru decizii non-critice (comentariu)